

Q.1 Write the simulation program to implement demand paging and show the page scheduling and total number of page faults according to the LFU page replacement algorithm. Assume the memory of n frames.
Reference String : 3,4,5,4,3,4,7,2,4,5,6,7,2,4,6

```
#include<stdio.h>
#define MAX 20
int frames[MAX],ref[MAX],mem[MAX][MAX],faults=0;
sp,m,n,count[MAX];
void accept()
{
    int i;
    printf("Enter no.of frames:");
    scanf("%d", &n);
    printf("Enter no.of references:");
    scanf("%d", &m);
    printf("Enter reference string:\n");
    for(i=0;i<m;i++)
    {
        printf("[%d]= ",i);
        scanf("%d",&ref[i]);
    }
}
void disp()
{
    int i,j;
    for(i=0;i<m;i++)
        printf("%3d",ref[i]);
    printf("\n\n");
    for(i=0;i<n;i++)
    {
        for(j=0;j<m;j++)
        {
            if(mem[i][j])
                printf("%3d",mem[i][j]);
            else
                printf(" ");
        }
        printf("\n");
    }
    printf("Total Page Faults: %d\n",faults);
}
int search(int pno)
{
    int i;
    for(i=0;i<n;i++)
    {
        if(frames[i]==pno)
            return i;
    }
    return -1;
}
int get_lfu(int sp)
{
    int i,min_i,min=9999;
    i=sp;
    do
    {
        if(count[i]<min)
        {
            min = count[i];
            min_i = i;
        }
        i=(i+1)%n;
    }while(i!=sp);
    return min_i;
}
void lfu()
{
    int i,j,k;
    for(i=0;i<m && sp<n;i++)
    {
        k=search(ref[i]);
        if(k==-1)
        {
            frames[sp]=ref[i];
            count[sp]++;
            faults++;
            sp++;
            for(j=0;j<n;j++)
                mem[j][i]=frames[j];
        }
        else
            count[k]++;
    }
    sp=0;
    for(i=0;i<m;i++)
    {
        k = search(ref[i]);
        if(k==-1)
        {
            sp = get_lfu(sp);
            frames[sp] = ref[i];
            count[sp]=1;
            faults++;
            sp = (sp+1)%n;
            for(j=0;j<n;j++)
                mem[j][i] = frames[j];
        }
        else
            count[k]++;
    }
}
int main()
{
    accept();
    lfu();
    disp();
    return 0;
}
```

Write the simulation program for demand paging and show the page scheduling and total number of page faults according the FIFO page replacement algorithm.

Assume the memory of n frames.

Reference String : 3, 4, 5, 6, 3, 4, 7, 3, 4, 5, 6, 7, 2, 4, 6

```
#include<stdio.h>
int frame[5][2],nf;
int searchFrames(int sv)
{
    int x;
    for(x=0;x<nf;x++)
    {
        if(sv==frame[x][0])
        {
            return 0;
        }
    }
    return 1;
}
void displayMemory()
{
    int i;
    printf("\n\nFrame Contains |");
    for(i=0;i<nf;i++)
    printf(" %d | ",frame[i][0]);
}

int findFreeFrame()
{
    int i,min=frame[0][1],ri=0;
    for(i=0;i<nf;i++)
    {
        if(frame[i][1]==-1)
        {
            return i;
        }
    }
    //LRU
    for(i=0;i<nf;i++)
    {
        if(min>frame[i][1])
        {
            min=frame[i][1];
            ri=i;
        }
    }
    return ri;
}

main()
{
    int rs[]={3, 4, 5, 6, 3, 4, 7, 3, 4, 5, 6, 7, 2, 4, 6};
    int ts=0;
    int n=15,i,j,pf=0,srch,insert_index;
    printf("Enter how many frames");
    scanf("%d",&nf);
    for(i=0;i<nf;i++)
    {
        for(j=0;j<2;j++)
        frame[i][j]=-1;
    }
    displayMemory();
    for(i=0;i<n;i++)
    {
        srch=searchFrames(rs[i]);
        if(srch==1)
        {
            pf++;
            insert_index=findFreeFrame();
            frame[insert_index][0]=rs[i];
            frame[insert_index][1]=ts++;
        }
        displayMemory();
    }
    printf("\n\nTotal Page Faults Occured is %d\n"
```

1:Write the simulation program to implement demand paging and show the page scheduling and total number of page faults according to the LRU (using counter method) page replacement algorithm. Assume the memory of n frames. Reference String : 3,5,7,2,5,1,2,3,1,3,5,3,1,6,2

```
#include<stdio.h>
int frame[5][2],nf;
int searchFrames(int sv)
{
    int x;
    for(x=0;x<nf;x++)
    {
        if(sv==frame[x][0])
        {
            return x;
        }
    }
    return -2;
}
void displayMemory()
{
    int i;
    printf("\n\nFrame Contains\n");
    for(i=0;i<nf;i++)
        printf(" %d | ",frame[i][0]);
}
int findFreeFrame()
{
    int i,min=frame[0][1],ri=0;
    for(i=0;i<nf;i++)
    {
        if(frame[i][1]==-1)
        {
            return i;
        }
    }
    for(i=0;i<nf;i++)
    {
        if(min>frame[i][1])
        {
            min=frame[i][1];
            ri=i;
        }
    }
    return ri;
}
main()
{
    int rs[]={3,5,7,2,5,1,2,3,1,3,5,3,1,6,2};
    int ts=0;
    int n=15,i,j,pf=0,srch,insert_index;
    printf("Enter how many frames");
    scanf("%d",&nf);
    for(i=0;i<nf;i++)
    {
        for(j=0;j<2;j++)
            frame[i][j]=-1;
    }
    displayMemory();
    for(i=0;i<n;i++)
    {
        srch=searchFrames(rs[i]);
        if(srch==-2)
        {
            pf++;
            insert_index=findFreeFrame();
            frame[insert_index][0]=rs[i];
            frame[insert_index][1]=ts++;
        }
        else
        {
            frame[srch][1]=ts++;
        }
        displayMemory();
    }
    printf("\n\nTotal Page Faults Occured is %d\n",pf);
}
```

Write the simulation program for demand paging and show the page scheduling and total number of page faults according the MFU page replacement algorithm. Assume the memory of n frames.

```
#include<stdio.h>
#define MAX 20
int frames[MAX],ref[MAX],mem[MAX]
[MAX],faults,
sp,m,n,count[MAX];
void accept()
{
    int i;
    printf("Enter no.of frames:");
    scanf("%d", &n);

    printf("Enter no.of references:");
    scanf("%d", &m);
    printf("Enter reference string:\n");
    for(i=0;i<m;i++)
    {
        printf("[%d]=",i);
        scanf("%d",&ref[i]);
    }
}
void disp()
{
    int i,j;
    for(i=0;i<m;i++)
        printf("%3d",ref[i]);
    printf("\n\n");
    for(i=0;i<n;i++)
    {
        for(j=0;j<m;j++)
        {
            if(mem[i][j])
                printf("%3d",mem[i][j]);
            else
                printf(" ");
        }
        printf("\n");
    }
    printf("Total Page Faults:
    %d\n",faults);
}
int search(int pno)
{
    int i;
    for(i=0;i<n;i++)
    {
        if(frames[i]==pno)
            return i;
    }
    return -1;
}
```

```
int get_mfu(int sp)
{
    int i,max_i,max=-9999;
    i=sp;
    do
    {
        if(count[i]>max)
        {
            max = count[i];
            max_i = i;
        }
        i=(i+1)%n;
    }while(i!=sp);
    return max_i;
}
void mfu()
{
    int i,j,k;
    for(i=0;i<m && sp<n;i++)
    {
        k=search(ref[i]);
        if(k!=-1)
        {
            frames[sp]=ref[i];
            count[sp]++;
            faults++;
            sp++;
            for(j=0;j<n;j++)
                mem[j][i]=frames[j];
        }
        else
            count[k]++;
    }
}
int main()
{
    accept();
    mfu();
    disp();
    return 0;
}
```

Write the simulation program for demand paging and show the page scheduling and total number of p according to the optimal page replacement algorithm. Assume the memory of n frames.

```
#include<stdio.h>
int main()
{
    int no_of_frames, no_of_pages,
    frames[10], pages[30],
    temp[10], flag1,
    flag2, flag3, i, j, k, pos, max,
    faults = 0;
    printf("Enter number of
    frames: ");
    scanf("%d", &no_of_frames);
    printf("Enter number of pages:
    ");
    scanf("%d", &no_of_pages);
    printf("Enter page reference
    string: ");

    for(i = 0; i < no_of_pages; ++i){
        scanf("%d", &pages[i]);
    }
    for(i = 0; i < no_of_frames; ++i){
        frames[i] = -1;
    }
    for(i = 0; i < no_of_pages; ++i){
        flag1 = flag2 = 0;
        for(j = 0; j < no_of_frames; ++j){
            if(frames[j] == pages[i]){
                flag1 = flag2 = 1;
                break;
            }
        }
        if(flag1 == 0){
            for(j = 0; j < no_of_frames; ++j){
                if(frames[j] == -1){
                    faults++;
                    frames[j] = pages[i];
                    flag2 = 1;
                    break;
                }
            }
        }
        if(flag2 == 0){
            flag3 = 0;
            for(j = 0; j < no_of_frames; ++j){
                temp[j] = -1;
                for(k = i + 1; k < no_of_pages; ++k){
                    if(frames[j] == pages[k]){
                        temp[j] = k;
                        break;
                    }
                }
            }
            for(j = 0; j < no_of_frames; ++j){
                if(temp[j] == -1){
                    pos = j;
                    flag3 = 1;
                    break;
                }
            }
            if(flag3 == 0){
                max = temp[0];
                pos = 0;
                for(j = 1; j < no_of_frames; ++j){
                    if(temp[j] > max){
                        max = temp[j];
                        pos = j;
                    }
                }
                frames[pos] = pages[i];
                faults++;
            }
            printf("\n");
            for(j = 0; j < no_of_frames; ++j){
                printf("%d\t", frames[j]);
            }
            printf("\n\nTotal Page Faults = %d", faults);
            return 0;
        }
    }
}
```

Write a program to implement the shell. It should display the command prompt "myshell\$". Tokenize the command line and execute the given command by creating the child process. Additionally it should interpret following 'list' commands as
myshell\$ list f dirname:- To print names of all the files in current directory.
myshell\$ list n dirname :- To print the number of all entries in the current directory

```
#include <sys/types.h>
#include <sys/stat.h>
#include <fcntl.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <dirent.h>
#include <unistd.h>
void make_toks(char *s, char
*tok[])
{
int i=0;
char *p;
p = strtok(s, " ");
while(p!=NULL)
{ tok[i++] = p;
p = strtok(NULL, " ");
}
tok[i] = NULL;
}
void list(char *dn, char op)
{
DIR *dp;
struct dirent *entry;
int dc=0,fc=0;
dp = opendir(dn);
if(dp==NULL)
{
printf("Dir %s not found.\n",dn);
return;
}

switch(op)
{
case 'f':
while(entry=readdir(dp))
{
if(entry->d_type==DT_REG)
printf("%s\n",entry->d_name);
} break;
case 'n':
while(entry=readdir(dp))
{
if(entry->d_type==DT_DIR) dc++;
if(entry->d_type==DT_REG) fc++;
}
printf("%d Dir(s)\t%d File(s)\n",dc,fc);
break;
case 'i':
while(entry=readdir(dp))
{
if(entry->d_type==DT_REG)
printf("%s\t%lu\n",entry->d_name,entry->d_fileno);
}
}
closedir(dp);
}

int main() {
char buff[80],*args[10];
int pid;
while(1)
{
printf("myshell$");
fflush(stdin);
fgets(buff,80,stdin);
buff[strlen(buff)-1]='\0';
make_toks(buff,args);
if(strcmp(args[0],"list")==0)
list(args[2],args[1][0]);
else
{
pid = fork();
if(pid>0)
wait();
else
{

```

Write a C program to implement the shell which displays the command prompt "myshell\$". It accepts the command, tokenize the command line and execute it by creating the child process. Also implement the additional command 'typeline' as typeline +n filename :- To print first n lines in the file.
typeline -a filename :- To print all lines in the file.

```
#include <sys/types.h>
#include <sys/stat.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <fcntl.h>
#include <dirent.h>
#include <unistd.h>
int make_toks(char *s, char
*tok[])
{
    int i = 0;
    char *p;
    p = strtok(s, " ");
    while(p != NULL)
    {
        tok[i++] = p;
        p = strtok(NULL, " ");
    }
    tok[i] = NULL;
    return i;
}
void typeline(char *op, char
*fn)
{
    int fh, i, j, n;
    char c;
    fh = open(fn, O_RDONLY);
    if(fh == -1)
    {
        printf("File %s not found.\n",
fn);
        return;
    }
    if(strcmp(op, "a") == 0)
    {
        while(read(fh, &c, 1) > 0)
            printf("%c", c);
        close(fh);
        return;
    }
    n = atoi(op);
    if(n > 0)
    {
        i = 0;
        while(read(fh, &c, 1) > 0)
        {
            printf("%c", c);
            if(c == '\n')
                i++;
            if(i == n)
                break;
        }
    }
    else if(n < 0)
    {
        i = 0;
        while(read(fh, &c, 1) > 0)
        {
            if(c == '\n') i++;
        }
        lseek(fh, 0, SEEK_SET);
        j = 0;
        while(read(fh, &c, 1) > 0)
        {
            if(c == '\n')
                j++;
            if(j == i+n+1)
                break;
        }
        while(read(fh, &c, 1) > 0)
        {
            printf("%c", c);
        }
        close(fh);
    }
}
int main()
{
    char buff[80], *args[10];
    while(1)
    {
        printf ("\n");
        printf("\nmyshell$ ");
        fgets(buff, 80, stdin);
        buff[strlen(buff)-1] = '\0';
        int n = make_toks(buff, args);
        switch (n)
        {
            case 1: if(strcmp(args[0], "exit") == 0)
                exit(1);
            if (!fork())
                execlp (args [0], args[0], NULL);
            break;
            case 2:
            if (!fork ())
                execlp (args [0],
args[0], args[1], NULL);
            break;
            case 3: if
                (strcmp(args[0],
"typeline") == 0)
                typeline (args[1],
args[2]);
            else
            {
                if (!fork ())
                    execlp (args [0],
args[0], args[1],
args[2], NULL);
            }
            break;
            case 4:
            if (!fork ())
                execlp (args [0], args
[0], args [1], args [2],
args [3], NULL);
            break;
        }
        return 0;
    }
}
```

:Write a program to implement the toy shell. It should display the command prompt "myshell\$". Tokenize the command line and execute the given command creating the child process. Additionally it should interpret the following commands.
count c filename :- To print number of characters in the file.
count w filename :- To print number of words in the file.
count l filename :- To print number of lines in the file.

```
#include <sys/types.h>
#include <unistd.h>
#include <sys/stat.h>
#include <fcntl.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
void make_toks(char *s, char
*tok[])
{
int i=0;
char *p;
p = strtok(s, " ");
while(p!=NULL)
{ tok[i++] = p;
p = strtok(NULL, " ");
}
tok[i] = NULL;
}
void count(char *fn, char op)
{ int fh, cc=0, wc=0, lc=0;
char c;
fh = open(fn, O_RDONLY);
if(fh == -1) {
printf("File %s not found.\n", fn);
return;
}
while(read(fh, &c, 1) > 0)
{ if(c == ' ')
wc++;
else if(c == '\n')
{
wc++;
lc++;
} cc++;
} close(fh);
switch(op)
{
case 'c':
printf("No. of
characters: %d\n", cc-1);
break;
case 'w':
printf("No. of words: %d\n", wc);
break;
case 'l':
printf("No. of lines: %d\n", lc+1);
break;
}
}
} int main()
{
char buff[80], *args[10];
int pid;
while(1) { printf("myshell$ ");
fflush(stdin);
fgets(buff, 80, stdin);
buff[strlen(buff)-1] = '\0';
make_toks(buff, args);
if(strcmp(args[0], "count") == 0)
count(args[2], args[1][0]);
else
{ pid = fork();
if(pid > 0)
wait();
else
{
if(execvp(args[0], args) == -1)
printf("Bad command.\n");
}
}
}
return 0;
}
```